CN Project 2 – chat log

Su and Claude, 4–6 November 2026

Su

login via claude browser and see this private repo: (I have access to it) https://github.com/milonithakkar/CS331-Phase2 and just check it out once (all the branches, most important Its is /0d one)

Claude

I looked through all three branches. Your browser was already signed in to GitHub, so I didn't have to log in.

The branches

- `main` has four commits from Oct 24. It holds only the planning docs: the proposal, the plan, the Project 1 audit and the README.
- `arena/29909b28` is an older single-commit attempt from Oct 25, co-authored by `arena-agent`. It's an all-Python toolkit (`fpstudy`) and it builds its pcaps with a synthetic generator (`gen_captures.py`). That conflicts with your rule that every limitation needs a real committed capture. Treat this branch as abandoned.
- `arena/0d29f32c` is the real one: 21 commits up to today (f5e8f8b). It uses C++20 for the QUIC work and Python for running and checking the experiments.

What 0d has

- QUIC v1 Initial extractor in C++, with `--hex` and `--pcap` modes. It covers header parsing and variable-length integers, key derivation checked against the RFC 9001 A.1 test vectors, header-protection removal and AES-GCM decryption, frame parsing and CRYPTO reassembly, and ClientHello parsing.
- Captures: `captures/manifest.json` lists 16 captures (the aioquic baseline plus Yash's 15 Track 1 captures), with metadata files and per-test docs T0–T7.
- QUIC result: the C++ extractor gets 5/5 ClientHellos from each QUIC capture (10/10 total), byte-identical to TShark 4.6.4.
- Impairment harness: `quic_impairment.py` tests reordering, shuffling, packet drops and truncation. The C++ tool matches TShark in every case.
- Bug fix (fec3b11): TShark's `jsonraw` output repeats the same key, so Python's `json.loads` was silently losing ClientHellos. This is fixed and has a test.
- Docs: `LEDGER.md` with L1–L7, `original-contribution-plan.md`, `verification-status.md`, a glossary, and viva and presentation guides.

Where it stands

1. No original contribution yet. The QUIC recovery idea was rejected by your own impairment results: there's no gap against TShark. What's left is an untested "diagnostics" idea, with L3 (JA4T/MSS from where the capture was taken) as the fallback. Both are blocked on missing evidence.
2. All seven limitations are still provisional: L1 is closed as a gap; L2 can't be reproduced because the key log isn't committed; L3's write-up doesn't match its saved numbers; L4 has one client in one environment; L5's cause still needs checking and it's already fixed upstream; L6's IPv6 pcap is empty (24 bytes); L7's DoH arm is incomplete.
3. Your WSL results aren't in the repo. The TShark comparison and impairment numbers are only marked "user-reported", because the raw exports, `impairment-results.json` and comparison script were never committed. That's the same reproducibility problem that cost marks in Project 1.
4. Every commit is authored by milonithakkar. That breaks the four-visible-authors rule, which was another Project 1 deduction.
5. The report and slides are empty.
6. The deadline is Fri Nov 6, 10:00 (commit time).

The cheapest high-value fixes are committing the WSL raw outputs and getting the other three teammates to commit something real. Picking the contribution is the harder call.

Su

https://github.com/YashG1836/CS331-T013-TLS_Fingerprint_2 meant this

Claude

I've gone through Yash's repo. It has only `main` (no other branches) and a single commit, b1ffd7c on Nov 3, authored by YashG1836 with claude as co-author: "Track 1: lab, real captures, tests T0–T7, results and docs".

What it is

This is Track 1, the measurement side. It runs the existing tools as black boxes (TShark 4.4.19 and FoxIO ja4 v0.18.6) inside a Docker lab on a MacBook on the campus network. It made 15 real captures and has one explanation file per test (T0–T7). One command, `make eval`, regenerates `results/summary.md`. There's no custom code beyond `capture.py` and `analyze.py`, plus 4 small tests.

What its LEDGER claims

ID	Claim	Main number
L1	QUIC: our guess was wrong, the tools can read it	5/5 hellos per capture, even split into 21 pieces

ID	Claim	Main number
L2	Certificate/HTTP fingerprints need TLS keys	JA4X+JA4H: 0 without keys, 5 with
L3	Where you capture changes what you see	same hello is 1 or 2 frames; MSS 65495 came from Docker's proxy
L4	Post-quantum key share	hello 508 → 1511 bytes; 12 vs 14 extensions
L5	One browser, many JA4s	Chromium: 15 JA4s in 50 hellos on released tools, 1 on unreleased FoxIO
L6	IPv4 vs IPv6	not measured, no IPv6 on the network
L7	ECH hides the name, DNS still shows it	DoH arm blocked

It also records three tool facts: FoxIO tag v0.18.6 reports itself as 0.18.5; FoxIO reads packets through TShark, so the two tools agreeing is not an independent check; neither command-line tool computes JA4T (in FoxIO's repo it only exists as a Zeek script).

The contribution candidates (decided Thu 5 Nov): the GREASE fix (already fixed upstream), adding JA4T/JA4TS to FoxIO's Rust tool with a warning for impossible SYN values, QUIC connection migration, and an ECH/DNS name-visibility table.

How it fits with Moloni's 0d branch

0d imported this whole repo in commit 04b530b and marked Yash's results as provisional, with these gaps: the T3 write-up doesn't match the saved numbers and the raw SYN rows aren't saved; the T2 key log isn't in the repo; `tls_ipv6.pcap` is empty but `summary.md` still shows `tools_agree=True`; the T7 ECH+DoH page output is empty even though the pcap has 12 hellos; the `ja4-main` "1 JA4" result in L5 has no raw output behind it.

So 0d is the newer combined version. Taken together, the strongest original contribution left is idea 2, JA4T plus the impossible-SYN warning, but only once it's backed by real SYN rows in a capture.

Other things I noticed

- The meeting transcript ends with Sameer sir focused on AI-use honesty: prompts must be raw. `ai/prompts.txt` exists, which is good, but the whole repo landing in one commit with Claude as co-author is weak evidence that four people did the work.
- The deadline is Fri Nov 6, 10:00, and the contribution is only being picked on Thu.

Su

please see if anything is wrong, will tell yash to do

Claude

I went through the scripts, the manifest, the raw JSON and the T2/T3/T4/T7 docs. Yash's work is solid overall: T4 already explains the padding effect and T7 already explains the DoH certificate. Below is what's still wrong or weak, most important first.

Wrong claims or numbers

1. T3/L3 doesn't show what the wire looks like. Docker Desktop's proxy ends the TCP connection itself (that's where the MSS 65495 comes from), so the Mac rebuilds the packets that actually reach Cloudflare. "1 frame vs 2 frames" only describes the container's own network interface.
2. T3 only turns off TSO/GSO, not GRO. `tls_offload_off` still has 2962-byte frames, bigger than the MTU. Fix: add `gro off` and re-capture.
3. The `biggest_frame` column is the largest frame in the whole pcap, including the server's responses, not the ClientHello. That's the "mismatch" Moloni's audit flagged.
4. The T3 SYN values (MSS 65495, TTL 63) aren't in any saved output, because `analyze.py` never asks for SYN fields.
5. T4's line "A fingerprint database must be rebuilt when clients move to PQ" isn't supported. The JA4 change comes from forcing a single group. Default curl offers both groups and keeps 13 extensions.
6. T4 says the PQ hello takes 2 TCP segments. In `tls_pq_on` (offload on) it's 1 frame.
7. L5's 50 Chromium hellos aren't all to the test site; the SNIs include accounts.google.com, update.googleapis.com and redirector.gvt1.com. Count JA4s per SNI.
8. L5's "ja4-main gives 1 JA4" can't be traced: `main_streams` are never written to `results/raw/`.
9. T7's ECH+DoH row is "-" but shouldn't be: 12 hellos to cloudflare-dns.com, and the ServerHello JA4S is `t130200_1301_a56c5b993250` vs Cloudflare's usual `..._234ea6891581`. The campus middlebox shows up in JA4S.
10. T7 uses `ja4-main` values while the other tests use the released tool.
11. The manifest says `quic_curl` is "ngtcp2 + OpenSSL", but `versions.txt` shows no ngtcp2. Probably OpenSSL's built-in QUIC.
12. `quic_chromium`'s SNI is `cloudflare-ech.com`, so Chromium used ECH over QUIC.

Code and reproducibility bugs

1. Neither script checks whether commands succeeded. Use `check=True` and log stderr.
2. For the empty IPv6 capture, `tools_agree` is True and `pieces_max` shows 1. Report N/A when there are 0 hellos.
3. The key log is gitignored, so re-running `make eval` turns L2's "5" into "-". It's safe to commit.
4. `seconds_tshark` and `seconds_ja4` change on every run, so `summary.md` never reproduces exactly.
5. `hello_bytes` leaves out the 4-byte header (1887/1505 vs C++ 1891/1509). Label the unit.
6. The manifest records no pcap hashes. Reuse Moloni's manifest.
7. The apt packages aren't pinned: Yash got TShark 4.4.19, Moloni's WSL has 4.6.4.
8. Only 4 tests, none touch `analyze_one`, and `make test` needs Docker.

Process

1. One commit, co-authored with claude. `ai/prompts.txt` must be the raw prompts, and the work should go in as several commits.
2. "Sameer sir meet_original.txt" includes the professor talking about marks. Better kept out of the submission.

The ones that matter most for the contribution decision are 1–4. Idea 2 (JA4T/JA4TS) can't be measured in this lab, because every SYN-ACK comes from Docker's proxy.

Su

can u also evalute miloni's? the /od one

Claude

The code on 0d is solid: bounds checks are careful, decryption is tested against the full RFC 9001 A.2 Initial packet, and its `analyze.py` already fixes Yash's silent-failure bug. The problems are mostly in how the results are tested and recorded, and in required items that don't exist yet.

Code problems

1. The "5 decrypt failures per capture" have a known cause. `main.cpp` derives Initial keys from each packet's own DCID. But once the server replies, the client switches to the server's chosen DCID, while Initial keys stay tied to the original DCID (RFC 9001 §5.2, RFC 9000 §7.2). So the client's later Initial packets always fail to decrypt: 5 original DCIDs plus 5 server-chosen ones = 10 flows. Any ClientHello piece re-sent after the server replied would be missed.
2. `parse_long_header_datagram` throws away the whole datagram if it hits a short-header packet or trailing bytes. Fix: return the packets parsed so far.
3. Two small RFC mismatches: the `0xc0` check skips grease_quic_bit clients (RFC 9287), and the frame parser accepts 0x1d in Initial packets (RFC 9000 §17.2.2).

Impairment test problems

1. The record selector doesn't filter by direction, so server Initials are included. "Drop median" and "truncate median" may have hit a server packet. Check `cases.json`.
2. The test can't find a recovery gap by design: the reference is TShark on the same damaged pcap, and removed bytes can't be recovered by any tool. Recall should be measured against the clean trace.
3. Almost every result is "user-reported". The raw files were written to the gitignored `results/reproduced/` folder and never committed.
4. `make eval` only checks hashes. It produces no measured numbers.
5. Yash's numbers come from TShark 4.4.19, yours from 4.6.4.

Missing project requirements

1. No `DECISIONS.md`.
2. No raw AI logs: `ai/` contains only a README.
3. No landscape doc (`docs/landscape.md`, `docs/basics/`).
4. The report and slides are empty.
5. Every commit is by milonithakkar.
6. Two zips were committed (04b530b, d71a402). They're still in git history; check for anything private.

Writing

1. The docs repeat the same caveats in four places and read as agent output ("the user reports", "not reproduced in Arena"). Rewrite in the team's own voice.

Coordination

Yash should make his fixes in the 0d branch, not his own repo. The most urgent before Friday: commit the raw outputs (6), fix the impairment selector and recall baseline (4–5), and add `DECISIONS.md` and the AI logs (9–10).

Su

we also have to make a presentation, could u tell me what each should contain? for the UI, i am thinking very basic, white background and black words, nothing esle. I want to properly go over it,

Claude

Here's a slide-by-slide outline: about 20 slides, roughly 15–20 minutes plus a short demo. Every result slide ends with a "does not prove" line.

Design: white background, black text, one plain sans font at large sizes, monospace for JA4 strings and commands, numbers as tables, one diagram for the QUIC pipeline.

Opening (1–5)

1. Title.
2. From Phase 1 to Phase 2: what Phase 1 built, its three problems, the professor's direction ("do QUIC"), the research question.
3. What a fingerprint is: the ClientHello, JA3 vs JA4, identifies a library not a user.
4. The fingerprint landscape: table of JA3, JA3S, JA4, JA4S, JA4T, JA4X, JA4H with layer, inputs, RFC section, needs keys, which tool. Tool facts underneath.
5. Method: Docker lab, one variable per capture, manifest with hashes, `make eval`, and the Docker proxy caveat up front.

The seven limitations (6–12)

Each with: Question, What we changed, Result, Why (RFC section), Does not prove.

1. L1 QUIC: 5/5, 21 fragments, RFC 9001 §5.2; our hypothesis was wrong.
2. L2 keys: 0 → 5 (4 certs + 1 HTTP), RFC 8446 §2, §4.4.2.
3. L3 capture point: 1586-byte frame vs 1514 + 138; MSS 65495 from Docker (RFC 9293 §3.7.1).
4. L4 PQ: 1511 vs 508 bytes, group 4588 vs 29; JA4 changes through side effects (RFC 8422 §5.1.2, RFC 7685).
5. L5: 50 JA3s, 15 JA4s released, 1 unreleased; GREASE (RFC 8701).
6. L6: not measured.
7. L7: ECH hides SNI, DNS still leaks; DoH blocked; middlebox visible in JA4S.
8. Summary table.

Depth dive A: QUIC (14–15)

1. The C++ pipeline diagram with RFC sections; RFC 9001 A.2 test vector; 10/10 match (only if committed).
2. Impairment matrix, why no gap was expected, the DCID-switch finding, the `jsonraw` harness bug; QUIC recovery rejected.

Depth dive B and contribution (16–17)

1. What a passive observer sees.
2. Our contribution (placeholder until the team decides).

Closing (18–21)

1. Reproducibility. 19. Limits and future work. 20. Who did what. 21. Conclusion with three takeaways.

Demo: run `quic-fingerprint --pcap captures/pcaps/quic_curl.pcap` and one TShark command showing the same bytes; keep a screenshot as backup.

Su

yes please do that, and make sure it's easy to understand! like no AI thingisies or anything like header/footer, prof cares about the understanding + simple tone rather than presentation

Claude

I've made the deck: 23 slides, black text on a white background, no headers, footers or page numbers. It's in plain language with one idea per slide. Every result slide uses the same four lines (what we changed, what we saw, why, what it doesn't prove), and each slide has speaker notes in simple words.

Placeholders to fill: team names on the cover and the "who did what" slide; the contribution slide, once the team decides; the L5 note's blank for Chromium's JA4 count on www.cloudflare.com only.

Lines that are only true once something is in the repo: slide 14's "all 10 ClientHellos matched TShark byte for byte" (needs the comparison files committed); slide 20's mention of `DECISIONS.md` and the AI logs (neither exists yet); slide 15's note about the "one packet removed" case (check `cases.json`).

Deck: https://claude.ai/artifact/AxVLvj5GxpdyKKPXJeAnxz

Su

so, nice format! can we also add some histroy? as to 1. how we got here? (one is connecting Phase 1 to phase 2) 2. connecting history (not a lot) but some and then build it like that? (so a narrative tone in the presentation, flowy)

Claude

I've added the history and changed the deck to tell a story. It's now 25 slides:

1. How we got here (slide 2, rewritten): Phase 1 told as a story, leading into Phase 2.
2. A short history (new): build, break, fix. 2009 Ivan Ristić shows browsers can be told apart from their handshake; 2017 JA3 from Althouse, Atkinson and Atkins at Salesforce; 2018 TLS 1.3 encrypts the certificate; 2021 QUIC; 2023 Chrome shuffles extensions, JA3 breaks, FoxIO answers with JA4; 2023–24 browsers turn on ECH and post-quantum key exchange.
3. Each change became one test (new): connects each history event to L1–L7.
4. Going deeper into QUIC (new): L1 worked on clean captures, so we built our own reader.
5. Conclusion (updated): ties back to the history.

Your earlier edit is kept: the team slide is still gone.

I wrote the history from what I know and didn't look the dates up. Worth checking before you present: the 2023 date for JA4, and the 2023–24 dates for ECH and post-quantum key exchange.

Deck: https://claude.ai/artifact/AxVLvj5GxpdyKKPXJeAnxz